Atria University · Neural Network and Convolutional Neural Network Sprint

Capstone Project · Group 5

Eyes in the Sky: Land-Use Classification from Satellite Images

CNN-based classification of Sentinel-2 satellite patches, with a data-efficiency study

No.	Member	GitHub	Role
1	Tejash	TejashRajuKV	Member (Part B)
2	Srivalli	Srivalli-DT	Member (Part A)
3	Rachana	rachana26paw	Member (Part C)
4	**Nitin (Team Leader)**	Nitinshakthi7	**Team Leader** (Part D)
5	Rohan	rohans-oss	Member (Part E)

Public repository: https://github.com/Nitinshakthi7/Eyes-in-the-Sky

Submitted 6 November 2026

Abstract

We built and compared convolutional neural networks (CNNs) for land-use classification on EuroSAT RGB: 27,000 Sentinel-2 patches of 64 × 64 pixels in 10 classes. All experiments use one fixed, stratified 70/15/15 split. Our own three-block CNN (94,986 parameters) reaches 92.72% test accuracy, averaged over three seeds. Transfer learning does better: a fully fine-tuned MobileNetV2 reaches 98.27% and ResNet-18 reaches 97.98%, while frozen pretrained backbones reach about 87%. A data-efficiency study shows the advantage of pretraining grows as labels shrink. With 5% of the training images, MobileNetV2 reaches 93.69% against 77.07% for our CNN. Error analysis shows that most mistakes fall among the agricultural classes, which are visually similar. Grad-CAM heat maps are coarse at this resolution, and a hiding control did not separate the highlighted regions from the rest, so we treat them as illustrations rather than proof. The models only know these 10 classes, and their accuracy drops sharply on images at other scales, so they should not be used beyond the EuroSAT setting.

1. Introduction and Motivation

Satellite imagery supports urban planning, deforestation monitoring, crop mapping, flood assessment and the study of city growth. Labelling land use by hand across large areas is not practical, so automatic classification with CNNs is a core tool in remote sensing. The question we study is not only whether a CNN can do this task, but how much labelled data it needs, whether a network designed by us can compete with a pretrained one, and what its mistakes and explanations tell us.

Our problem: classify 64 × 64 RGB patches into 10 land-use / land-cover classes. We compare a scratch CNN with transfer learning, run ablations on the scratch CNN, study accuracy as the training set shrinks from 100% to 5%, and analyse errors with a confusion matrix, Grad-CAM and t-SNE.

2. Related Work

- **EuroSAT (Helber et al., 2019).** Introduced the dataset we use: Sentinel-2 patches labelled into 10 classes, with a benchmark of several CNNs. It sets the common reference for this task.
- **ResNet (He et al., 2016).** Residual (skip) connections let very deep networks train well. We use ResNet-18, pretrained on ImageNet, as one transfer model.
- **MobileNetV2 (Sandler et al., 2018).** Uses inverted residual blocks and depthwise separable convolutions to get good accuracy with few parameters. We use it as our second transfer model; it is our best model.
- **Grad-CAM (Selvaraju et al., 2017).** Shows which image regions push a CNN's prediction, using gradients of the class score with respect to the last convolutional feature maps. We implemented it by hand.
- **t-SNE (van der Maaten and Hinton, 2008).** Projects high-dimensional features to 2-D so that similar points stay close. We use it to check whether the learned features group by class.

3. Dataset

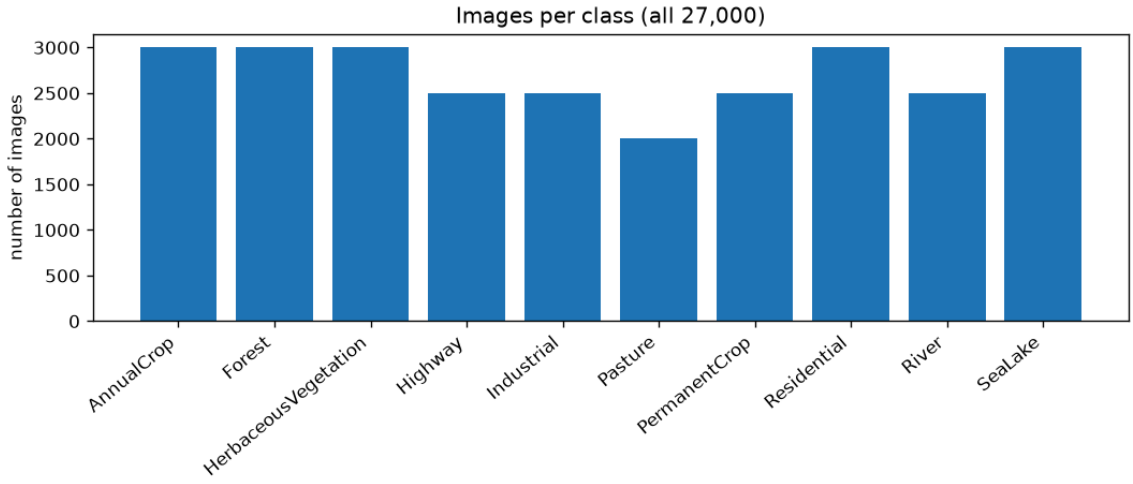
Source and content. We use the RGB version of EuroSAT: 27,000 labelled 64 × 64 JPEG patches from Sentinel-2 at about 10 metres per pixel, in 10 classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake. The data is public and contains no people or personal information.

Split. `make_split(seed=42)` applies scikit-learn's `train_test_split` with `stratify` twice: 70% train, then the remaining 30% split 50/50 into validation and test. The result is saved as `data/splits.csv`, which is committed to git, so every member uses exactly the same images. The class counts below are from the split file.

Class	Total	Train	Validation	Test
AnnualCrop	3,000	2,100	450	450
Forest	3,000	2,100	450	450
HerbaceousVegetation	3,000	2,100	450	450
Highway	2,500	1,750	375	375
Industrial	2,500	1,750	375	375
Pasture	2,000	1,400	300	300
PermanentCrop	2,500	1,750	375	375
Residential	3,000	2,100	450	450

Class	Total	Train	Validation	Test
River	2,500	1,750	375	375
SeaLake	3,000	2,100	450	450
Total	**27,000**	**18,900**	**4,050**	**4,050**

Exploratory analysis and leakage checks. The EDA notebook (`notebooks/01_eda.ipynb`) shows the class counts, one sample grid per class, per-class mean colour and per-channel histograms (Figure 1). Classes are balanced to within a factor of 1.5 (2,000 to 3,000 images), so accuracy and macro-F1 do not differ much. Before training, we check that no file path appears in two splits and that each split keeps the class shares. We also compare MD5 file hashes to catch identical images saved under different names.



Preprocessing and augmentation. Every model receives pixels scaled to 0–1 and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). For training images only, we apply random rotations of 90, 180 or 270 degrees and horizontal or vertical flips. A satellite tile has no "up" direction, so these changes keep the label. Validation and test images are never augmented.

Caveat. The split is random over patches. Neighbouring patches from the same area can therefore fall in both training and test sets, so the error on completely new regions may be higher than our 1.7%.

4. Methodology

Scratch CNN. The baseline has three blocks. Each block is a 3 × 3 convolution, batch normalisation, ReLU and 2 × 2 max pooling, with 32, 64 and 128 filters. Global average pooling reduces each map to one number, and a linear layer gives 10 class scores.

```
RGB input (3 × 64 × 64)
  → Conv 3×3 (32)  → BatchNorm → ReLU → MaxPool 2×2   (32 × 32 × 32)
  → Conv 3×3 (64)  → BatchNorm → ReLU → MaxPool 2×2   (64 × 16 × 16)
  → Conv 3×3 (128) → BatchNorm → ReLU → MaxPool 2×2   (128 × 8 × 8)
  → Global average pool                                (128)
  → Linear (128 → 10)                                   → class scores
Total parameters: 94,986
```

We chose three blocks because the spatial size shrinks 64 → 32 → 16 → 8, enough to capture local texture and a wider context without overfitting a 19k-image training set. Batch normalisation stabilises training. Global average pooling replaces a large dense layer, which keeps the parameter count low.

Ablations. Each ablation changes one design choice, so its effect can be read directly:

Setting	Change from baseline
Baseline	3 blocks, BatchNorm on
No BatchNorm	BatchNorm removed from all blocks
Two blocks	Third block removed (32 and 64 filters only)

Transfer models. ResNet-18 and MobileNetV2 start from ImageNet weights. We replace the final classifier with a new 10-way layer (`model.fc` for ResNet-18, `model.classifier[1]` for MobileNetV2). In **frozen** mode only the new layer trains (learning rate 0.001). In **full fine-tuning** all weights train (learning rate 0.0001, lower so the pretrained features are not destroyed).

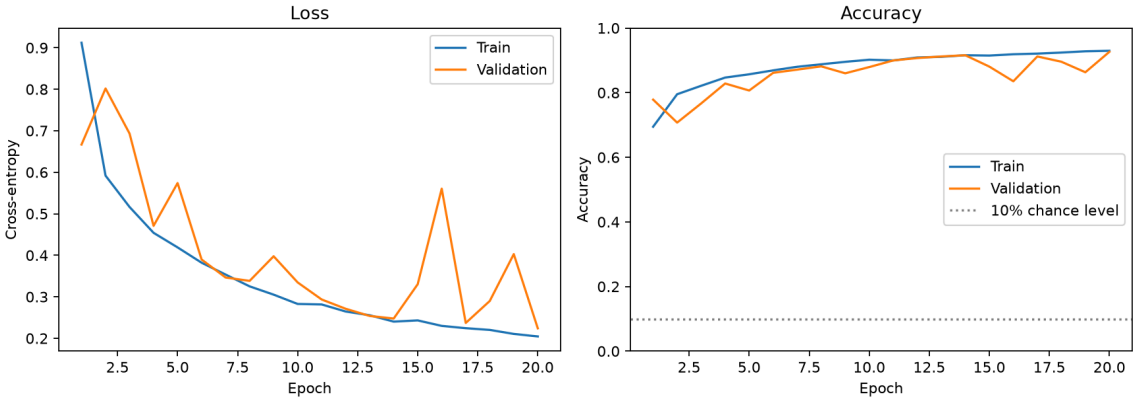
Training recipe (same for every model). Adam optimiser, cross-entropy loss, batch size 64, at most 20 epochs. The epoch with the best validation accuracy is kept. The test set is scored once per finished run and never used to make any decision. Each setting runs with seeds 0, 1 and 2, and we report the mean and sample standard deviation (ddof = 1).

5. Experiments and Results

Main comparison. Test set, 4,050 images, models trained on all 18,900 training images, mean ± sample standard deviation over three seeds (`results/metrics.csv`).

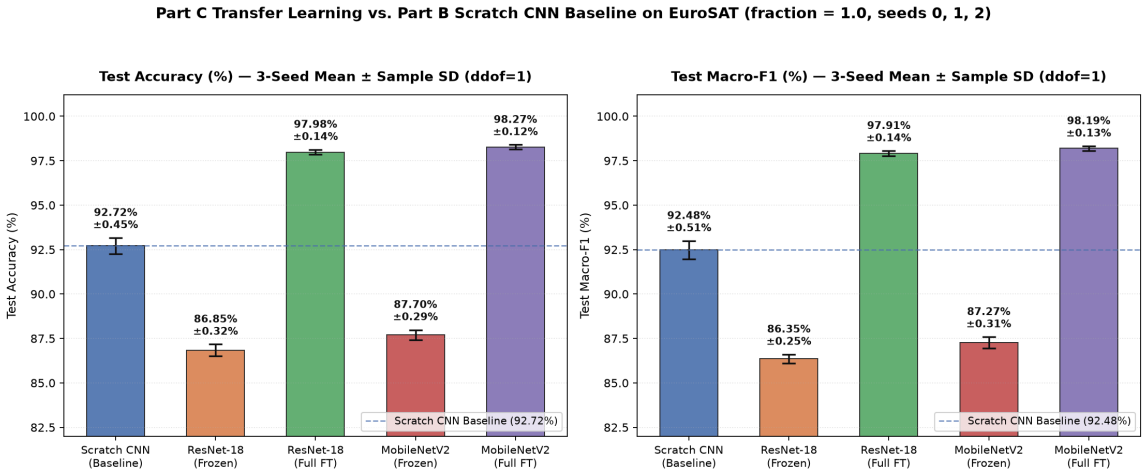
Model	Test accuracy	Test macro-F1
Scratch CNN baseline	92.72% ± 0.45%	92.48% ± 0.51%
Scratch CNN, no BatchNorm	88.95% ± 0.63%	88.45% ± 0.64%
Scratch CNN, two blocks	85.31% ± 1.43%	84.18% ± 1.78%
ResNet-18, frozen	86.85% ± 0.32%	86.35% ± 0.25%
MobileNetV2, frozen	87.70% ± 0.29%	87.27% ± 0.31%
ResNet-18, full fine-tuning	97.98% ± 0.14%	97.91% ± 0.14%
MobileNetV2, full fine-tuning	**98.27%± 0.12%**	**98.19%± 0.13%**

Ablations. Removing BatchNorm costs 3.8 points of accuracy and 4.0 points of macro-F1. The two-block model costs 7.4 points of accuracy and has the largest seed spread (± 1.43). Both drops are larger than the seed-to-seed spread of the baseline (± 0.45), so the components do matter for this setup. Because we only have three seeds, these are evidence, not proof.



The learning curves (Figure 2) show training and validation accuracy both rising over 20 epochs and ending close together. We see no large widening gap, so overfitting is limited. Validation accuracy moves up and down between epochs, which is why the best-validation epoch is chosen rather than the last one.

Transfer learning. Full fine-tuning beats frozen backbones by about 10 points for both models. The two modes also use different learning rates, so this comparison mixes two changes. We keep it because the team used the same recipe for every model, and the caveat is stated here. Both pretrained models beat the scratch CNN by 5 to 6 points; the gap between ResNet-18 and MobileNetV2 (0.3 points) is too small to call.



Data efficiency. We trained the scratch CNN and MobileNetV2 (full fine-tuning) on 5%, 10%, 25%, 50% and 100% of the training images. The validation and test sets stay the same. Results are mean ± sample standard deviation over seeds 0, 1 and 2, except where noted in the table.

Training images	Scratch CNN	MobileNetV2 fine-tuned	Gap
945 (5%)	77.07% ± 0.74%	93.69% ± 0.20%	16.6
1,890 (10%)	80.98% ± 1.20%	96.15% ± 0.33%	15.2
4,725 (25%)	84.96% ± 1.86%	97.26% ± 0.09%	12.3
9,450 (50%)	89.45% ± 1.20%	97.76% ± 0.01%	8.3
18,900 (100%)	92.72% ± 0.45%	98.27% ± 0.12%	5.6

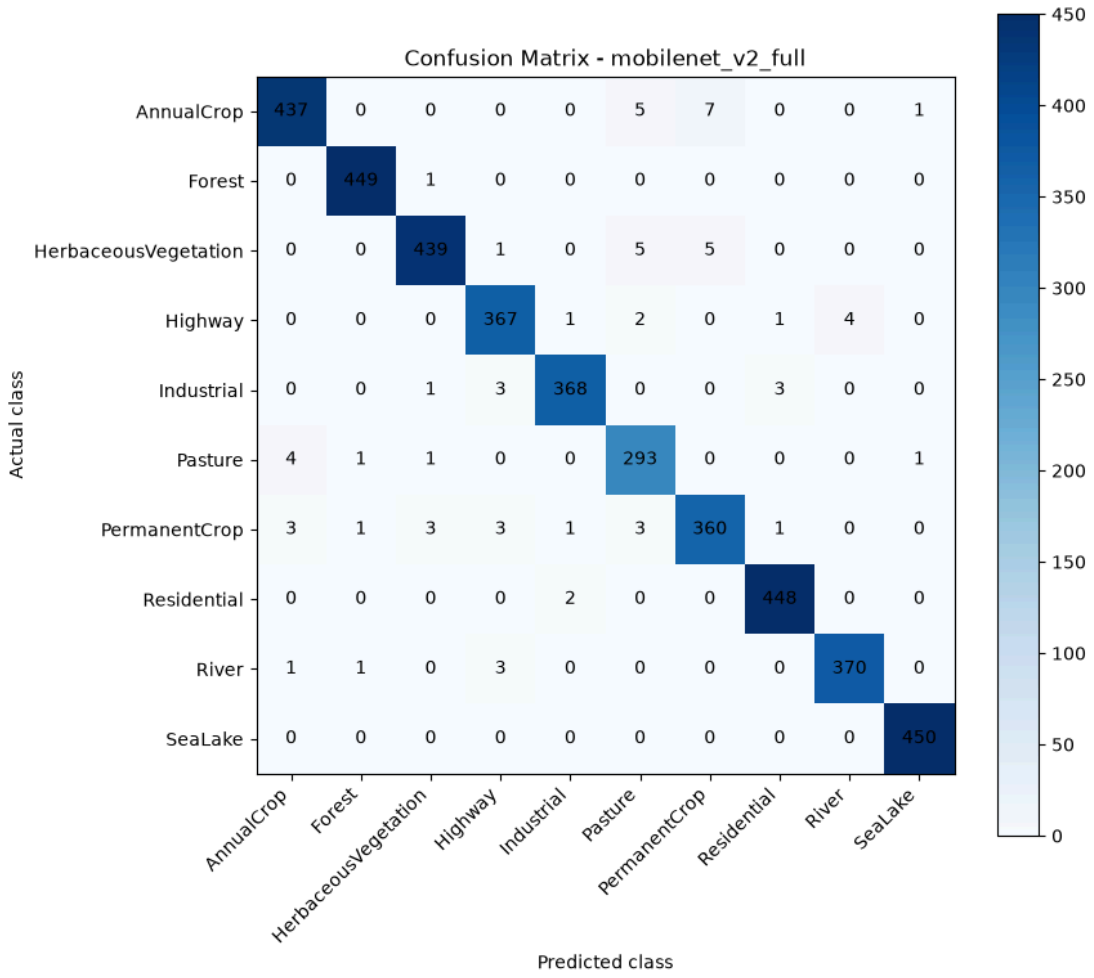


The pretrained model with 945 labels is already better than our CNN with all 18,900. The gap shrinks as data grows but never closes: pretraining mainly helps when labels are scarce. Rohan's extra seed-4 runs of the scratch CNN at 5%, 10% and 25% are in `metrics.csv` and are not averaged into the table above.

6. Error Analysis and Explainability

Confusion matrix. The best model (MobileNetV2, seed 0, evaluated on the CPU) gets 3,981 of 4,050 test images right, 98.3%, with 69 mistakes. The GPU run of the same weights gives 3,980 (98.27%), which

is the value in the table above. Most classes are almost perfect: SeaLake 450/450, Forest 449/450, Residential 448/450.



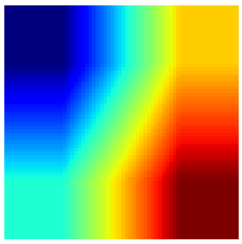
Where the mistakes are. 36 of the 69 mistakes (52%) are among AnnualCrop, PermanentCrop, Pasture and HerbaceousVegetation. The largest single confusion is 7 AnnualCrop tiles predicted as PermanentCrop. Other frequent pairs are AnnualCrop → Pasture (5), HerbaceousVegetation → Pasture (5), HerbaceousVegetation → PermanentCrop (5) and Highway → River (4). Visually, these classes share green fields and similar textures, and Highway and River both contain long, narrow structures. We looked at examples, but we did not test the cause directly. PermanentCrop has the lowest recall of any class (360 of 375, 0.960).

Grad-CAM. Grad-CAM weights the last convolutional feature maps by the average gradient of the predicted class score, sums them, and keeps the positive part (ReLU) as a heat map. We wrote it by hand with a forward hook (to keep the activations) and a backward hook (to keep the gradients). Because MobileNetV2's last block is only 2 × 2 cells on a 64 × 64 tile, the maps are coarse. The figure shows one correct and one wrong example per class; the selection is done by the script in notebook 04, not by hand.

AnnualCrop - Wrong



Grad-CAM



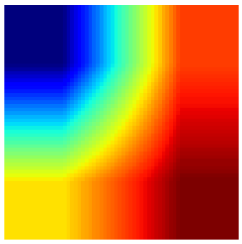
Predicted: PermanentCrop



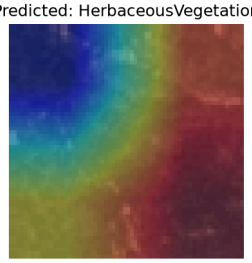
Forest - Wrong



Grad-CAM



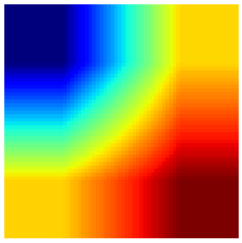
Predicted: HerbaceousVegetation



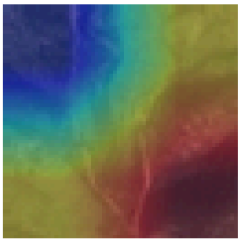
HerbaceousVegetation - Wrong



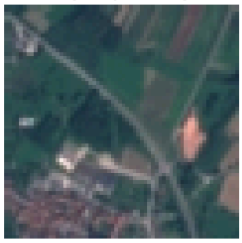
Grad-CAM



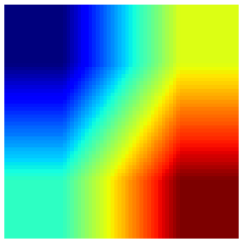
Predicted: Highway



Highway - Wrong



Grad-CAM



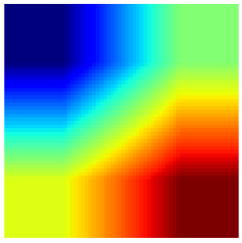
Predicted: Pasture



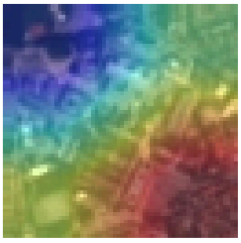
Industrial - Wrong



Grad-CAM



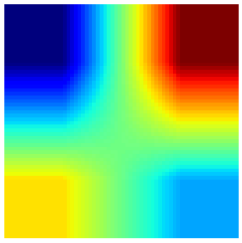
Predicted: Residential



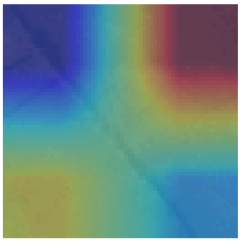
Pasture - Wrong



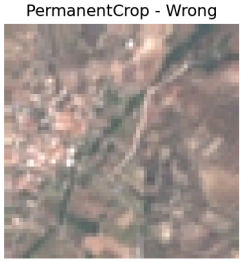
Grad-CAM



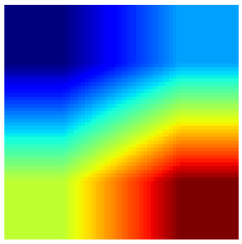
Predicted: SeaLake



PermanentCrop - Wrong



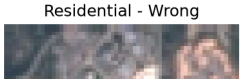
Grad-CAM



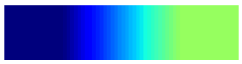
Predicted: Residential



Residential - Wrong

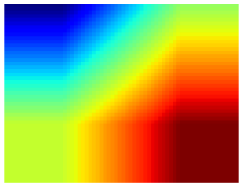


Grad-CAM

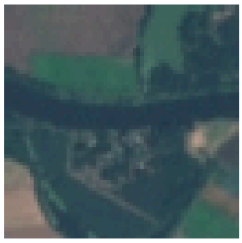
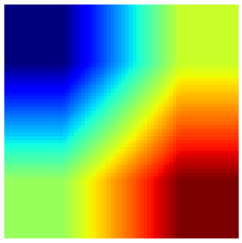
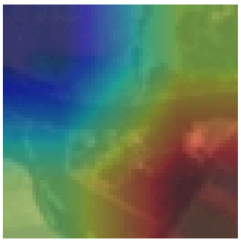


Predicted: Industrial

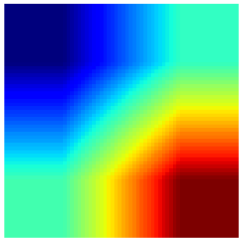
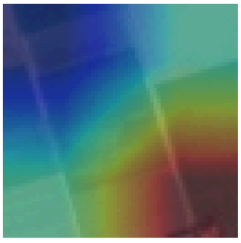


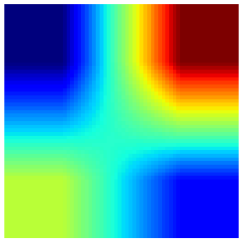
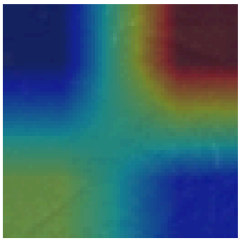

River - Wrong | Grad-CAM | Predicted: AnnualCrop

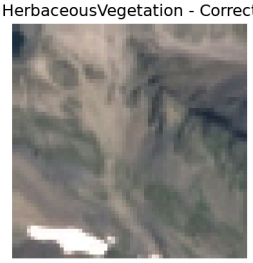
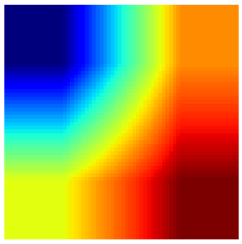
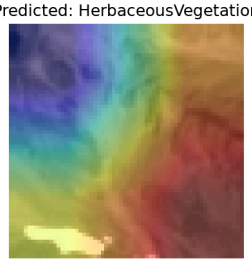
AnnualCrop - Correct | Grad-CAM | Predicted: AnnualCrop

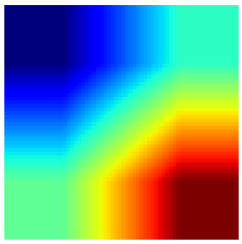
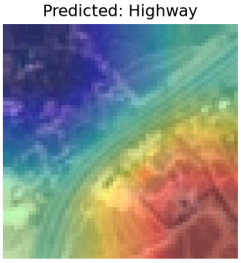
Forest - Correct | Grad-CAM | Predicted: Forest

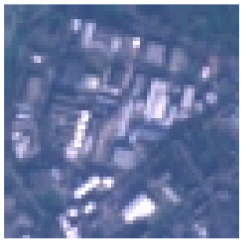
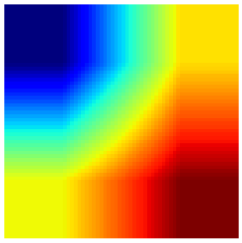
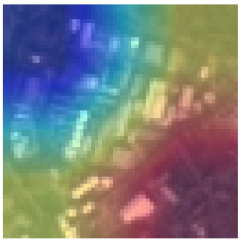
HerbaceousVegetation - Correct | Grad-CAM | Predicted: HerbaceousVegetation

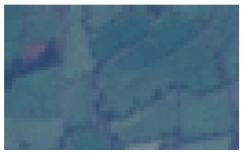
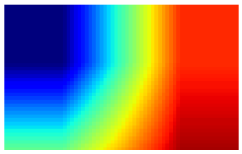
Highway - Correct | Grad-CAM | Predicted: Highway

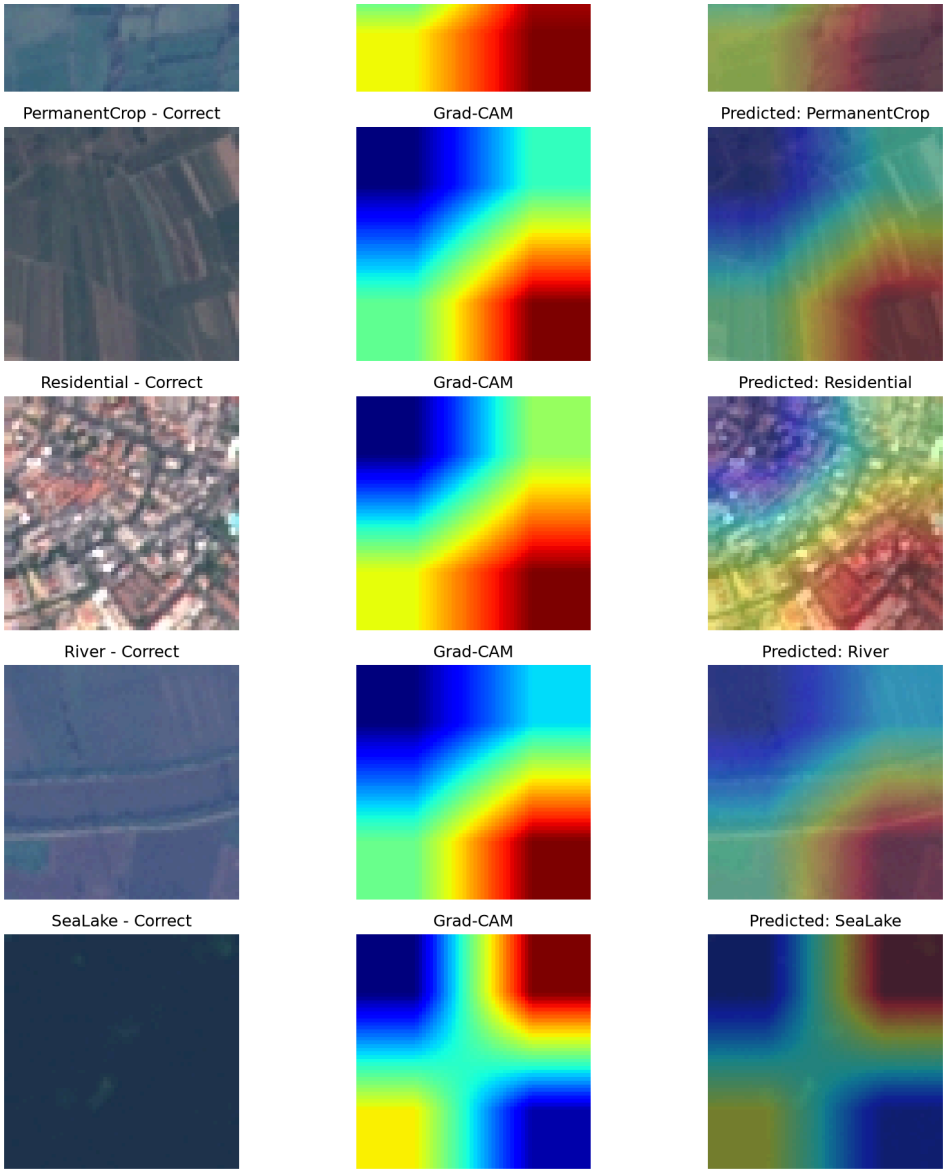




Industrial - Correct | Grad-CAM | Predicted: Industrial

Pasture - Correct | Grad-CAM | Predicted: Pasture

Does Grad-CAM show what matters? We tested it with a control. On 200 images, hiding the hottest 25% of each tile reduced confidence by 0.064 on average. Hiding the coldest 25% reduced it by 0.061. The two are almost the same, so in this experiment the heat map did not identify the pixels the model depends on. The hottest quarter was in the bottom-right corner in 80% of images, which suggests a positional bias in the explanation method at this resolution. We therefore present Grad-CAM as a visual aid, not as proof of what the model uses.

t-SNE. We took the MobileNetV2 features just before the final layer for about 2,000 random test images (fixed seed, perplexity 30) and plotted them coloured by class (Figure 7). Classes that the confusion matrix rarely mixes form separate groups, while the agricultural classes overlap, which matches Section 6. t-SNE distances between groups are not measurements, and the plot does not show how the model reaches a decision.



7. Limitations, Ethics and Future Work

Only EuroSAT-style tiles. The models learned from 64 × 64 Sentinel-2 tiles covering about 640 m. In a check, when the tile showed half that width (about 320 m), MobileNetV2's accuracy fell from 98% to about 36%, and to about 13% at a quarter of the width. Part of that drop comes from blurring when the crop is enlarged, but it shows the models depend strongly on scale.

Always one of 10 classes. There is no "none of these" output. A plain red square was called SeaLake with 100% confidence by both the MobileNetV2 and ResNet-18 models. Mistakes also come with fairly high confidence (76% on average), so confidence does not warn the user reliably.

Random split. As noted in Section 3, neighbouring patches can be in both training and test sets, so accuracy on new regions may be lower.

Confounded comparison. Frozen and fully fine-tuned models differ in learning rate as well as in which weights train. BatchNorm statistics also keep updating in the "frozen" models.

Small numbers. Three seeds per setting, and 69 test mistakes: a difference of four against five mistakes is noise.

Ethics. EuroSAT is a public, anonymised dataset of land-cover tiles. It contains no people or personal information. The models must not be used as a basis for decisions about individual land or people without expert review. Our demo website is a teaching tool and is not a general satellite-image reader.

Future work. Test the models on other regions and seasons (a geographic hold-out split). Use the 13-band multispectral version of EuroSAT. Add a "none of these" or out-of-distribution check. Use UMAP alongside t-SNE. Repeat the data-efficiency study with more seeds at the smallest sizes.

8. Conclusion

A CNN designed by us reaches 92.7% on EuroSAT, and each design choice we tested (BatchNorm, depth) matters by several points. Transfer learning reaches 98.3%, and the gain is largest when labels are scarce: with 5% of the labels, a pretrained MobileNetV2 is already better than our CNN with all of them. Errors concentrate in visually similar agricultural classes. Grad-CAM and t-SNE help us see the model's behaviour, but our control shows that Grad-CAM heat maps must be read with care. The main limit is the scope: the models only work well on the kind of tile they were trained on.

9. Use of AI Tools

We used AI assistants in this project and disclose it here as the brief requires. Claude Code (Nitin) generated the repository skeleton and the laptop check script, wrote the shared training function `train()` and the data-efficiency notebook, built the website server in `app/server.py`, and reviewed teammates' pull requests. Tejash used Codex to implement the scratch CNN and its ablations. Rachana used ChatGPT and Gemini for setup help and for explaining transfer-learning concepts. Srivalli used Claude Code to write the data code and report drafts, and Copilot for the part-A viva answers. Rohan used ChatGPT for the evaluation, Grad-CAM and t-SNE work. We checked AI output by running the code, reading every line, comparing outputs with the saved results, and re-scoring the checkpoints (for example, Tejash's nine checkpoints were reloaded and re-scored and matched his rows exactly). Every member can explain the code in their part. The full log, with one row per use and what was checked, is in `Backend/docs/ai_usage.md`.

10. Contribution Table

Member	GitHub	Responsibilities	Key files and experiments
Nitin (team leader)	Nitinshakthi7	**Part D.** Repository, team plan, shared `train()`, data-efficiency study (24 runs), Grad-CAM control, website server, pull-request review, this report's assembly	`src/utils.py`, `src/train.py`, `notebooks/05_data_efficiency.ipynb`, `check_setup.py`, `app/server.py`, `src/gradcam_sanity_check.py`
Srivalli	Srivalli-DT	**Part A.** Data loading, fixed stratified split, leakage checks, augmentation, EDA	`src/dataset.py`, `data/splits.csv`, `notebooks/01_eda.ipynb`

Member	GitHub	Responsibilities	Key files and experiments
Tejash	TejashRajuKV	**Part B.** Scratch CNN, ablations (BatchNorm, depth), learning curves, website pages	`src/models.py` (scratch CNN), `notebooks/02_baseline_cnn.ipynb`, `frontend/`
Rachana	rachana26paw	**Part C.** Transfer models (ResNet-18, MobileNetV2; frozen and full), transfer comparison	`src/models.py` (transfer models), `notebooks/03_transfer_learning.ipynb`
Rohan	rohans-oss	**Part E.** Evaluation, confusion matrix, Grad-CAM, Grad-CAM control, t-SNE, seed-4 runs	`src/evaluate.py`, `src/gradcam.py`, `src/visualize_*.py`, `notebooks/04_evaluation_and_explainability.ipynb`

Every member's work is in the repository history through pull requests.

11. References

1. Helber, P., Bischke, B., Dengel, A., Borth, D. (2019). EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7), 2217–2226.
2. He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*, 770–778.
3. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR*, 4510–4520.
4. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV*, 618–626.
5. van der Maaten, L., Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.
6. Sentinel-2 mission, European Space Agency (Copernicus programme).
7. PyTorch and torchvision (models and pretrained ImageNet weights); scikit-learn (splits, metrics, t-SNE).

Appendix A: Reproducing the results

Full steps are in the repository README. In short (inside the `Backend/` folder, with the virtual environment on):

```
python check_setup.py --name YourName --download   # checks the laptop; downloads data and weights
python -m src.utils                                 # seed and device check
python -c "from src.train import train; train('mobilenet_v2_full', 1.0, 0)"
python -m src.evaluate --model mobilenet_v2_full    # confusion matrix and mistakes
python -m app.server                                # the website, at http://127.0.0.1:8000/
```

Every run appends one row to `results/metrics.csv`. Fixed seeds, the fixed split in `data/splits.csv`, and the library versions in `requirements.txt` make the results repeatable. Trained weights are not in git because of their size; they are shared as a download link in `Backend/models/README.md`.

Appendix B: Repository layout

`README.md` (overview, setup, results, contribution table) · `Backend/src/` (data, models, training, evaluation, Grad-CAM) · `Backend/notebooks/` (01 EDA, 02 scratch CNN, 03 transfer learning, 04 evaluation and explainability, 05 data efficiency) · `Backend/results/` (metrics and figures) · `Backend/app/` (website server) · `Backend/docs/` (plan, member logs, viva notes, AI usage) · `frontend/` (website pages).